

Prompt-Feature Activation in Large Language Models

*A Mechanistic Interpretability Study Using
Sparse Autoencoders on GPT-2 Small*

CS 463 — Machine Learning
Final Project

Ankit Mukhopadhyay

University of San Francisco

Spring 2026

Model: GPT2 | SAE: gpt2-small-res-jb | Features: 24,576 | Layers: [2, 6, 10]

Table of Contents

- 1.** Abstract
- 2.** Scope Changes from Milestone 1
- 3.** Research Question
- 4.** Background & Key Concepts
- 5.** Methodology & Pipeline
- 6.** Notebook 0 — Data Preparation & EDA
- 7.** Notebook 1 — Environment Setup & Model Loading
- 8.** Notebook 2 — Feature Extraction
- 9.** Notebook 3 — Prompt Analysis & Visualization
- 10.** Notebook 4 — Activation Patching & Causal Verification
- 11.** Key Observations & Results
- 12.** Discriminative Feature Analysis
- 13.** Reflection & Discussion
- 14.** Conclusions & Future Work
- 15.** Companion Documents

1. Abstract

This project investigates how different categories of natural language prompts — math, code, factual, creative, emotional, and reasoning — activate distinct sets of interpretable features inside a language model's residual stream. We use GPT-2 Small (12 layers, 768 dimensions, 12 attention heads) paired with pre-trained Sparse Autoencoders (SAEs) from the gpt2-small-res-jb release (24,576 learned features per layer) to decompose the model's internal representations into monosemantic features.

Our analysis pipeline spans four notebooks: environment setup with automatic model selection, feature extraction across three target layers (2, 6, 10), statistical and visual analysis of feature activation patterns, and causal verification through activation patching. We find that prompt categories produce measurably different feature activation signatures, with the strongest separability at early layers (Fisher ratio = 1.33 at layer 2). Creative prompts activate the most features ($L0 \approx 332$ at layer 10), while math and reasoning prompts are the sparsest ($L0 \approx 10-15$ at layer 2). Activation patching confirms that later layers (especially layer 11) are most causally important for output generation, though the math-relevant information appears distributed across many features rather than concentrated in a few.

2. Scope Changes from Milestone 1

The project scope evolved meaningfully from the Milestone 1 proposal in response to instructor feedback, hardware constraints, and accumulated practical knowledge. We document four major changes here for transparency.

Change 1 — Model: Gemma 2 2B → GPT-2 Small

Originally proposed Gemma 2 2B with Gemma Scope SAEs. Switched to GPT-2 Small (124M) for two reasons: (1) hardware constraint — Gemma 2 2B cannot fit comfortably on our RTX 3050 Ti's 4.3 GB VRAM alongside activation caches, and (2) the gpt2-small-res-jb SAE release by Joseph Bloom is a more battle-tested, well-documented pre-trained SAE than Gemma Scope at the time of this work. The methodology transfers cleanly — only the model swap was needed.

Change 2 — Data: 120 Synthetic → 300 Real Human Prompts

Milestone 1 used 120 hand-curated synthetic prompts (20 per category). Per instructor feedback that synthetic prompts 'muddle experimental exploration' (they reflect researcher intuitions rather than real user behavior), we replaced them with 300 real human prompts (50 per category) sourced from Search Arena 24k and WildChat 4.8M — both authentic, production-system datasets.

Change 3 — Pipeline: Added Comprehensive EDA Notebook (NB00)

A new Notebook 0 was added covering data acquisition (HuggingFace streaming for WildChat's 27 GB), category mapping strategies (with sub-classification logic for ambiguous topics like 'tutoring_or_teaching'), a multi-stage quality filtering pipeline (length, alpha ratio, type-token ratio, GPT-2 token limit, deduplication), and 6 EDA visualizations. This notebook is documented in detail in Section 6.

Change 4 — Focus: Top-K Discriminative Feature Analysis

Most recent professor feedback recommended narrowing the analytical focus from "all 24,576 features" to a small, deeply-analyzed set of the most discriminative features per category. Section 13 (Future Work) elaborates on this direction — analyzing the top ~5 features per category with Neuronpedia annotations, manual inspection of activating examples, and direct feature-level steering experiments.

3. Research Question

Primary Question:

"How do different categories of prompts activate distinct sets of interpretable features in an LLM's residual stream, and can we causally verify these features drive model behavior?"

Sub-questions:

- Do math, code, factual, creative, emotional, and reasoning prompts produce different feature activation patterns?
- At which layers do these differences emerge most strongly?
- How sparse are feature activations, and does sparsity vary by prompt type?
- Can we causally verify that specific features drive category-specific behavior via activation patching?
- Can we steer model output by amplifying category-specific feature directions?
- Do findings on 300 real human prompts diverge from results on 120 synthetic prompts (per the M1→M2 dataset shift)?

4. Background & Key Concepts

Superposition:

Neural networks encode more concepts (features) than they have dimensions. Individual neurons are polysemantic — they respond to multiple unrelated concepts. This makes interpreting individual neurons unreliable.

Sparse Autoencoders (SAEs):

SAEs decompose a model's d-dimensional activation vector into a much larger (e.g., 24,576) set of monosemantic features. The key insight: while the original representation is dense and entangled, the SAE latent space is sparse — only ~50-300 features are nonzero for any given input. Each feature ideally corresponds to one interpretable concept.

Feature Activation & Sparsity (L0):

L0 measures how many SAE features have nonzero activation for a given input. Low L0 means highly sparse (few features active). We track L0 across prompt categories and layers to understand activation density differences.

Residual Stream:

The central 'highway' of information flow in a transformer. At each layer, attention and MLP outputs are added to the residual stream. SAEs are trained on residual stream activations at specific hook points (hook_resid_pre for GPT-2).

Activation Patching:

A causal intervention technique: run the model on two different inputs, then swap internal activations from one run into the other at specific layers. By measuring how much the output changes (via KL divergence), we identify which layers and features are causally important for generating category-specific outputs.

Feature Steering:

Amplifying specific SAE feature directions in the residual stream to steer model output toward a desired category (e.g., making a creative prompt produce math-like output).

5. Methodology & Pipeline

5.1 Model & SAE Selection

We selected GPT-2 Small (124M parameters, 12 layers, $d_{\text{model}}=768$) as our target model for two complementary reasons. First, hardware compatibility: our development machine (RTX 3050 Ti with 4.3 GB VRAM) cannot fit Gemma 2 2B at full precision, but GPT-2 Small at float32 is only ~500 MB — leaving headroom for activation caching and patching experiments. Second, and more importantly, the gpt2-small-res-jb SAE release by Joseph Bloom (EleutherAI) provides high-quality pre-trained sparse autoencoders for every layer of GPT-2 Small, with 24,576 learned features per layer (a $32\times$ expansion of d_{model}). This gives us interpretable feature decompositions without needing to train SAEs from scratch — which would itself be a multi-week research project. The SAEs are loaded via the `sae_lens` library.

5.2 Dataset: 300 Real Human Prompts

5.2.1 Why Real Data?

Milestone 1 used 120 hand-curated synthetic prompts (20 per category). Per instructor feedback, self-generated prompts risk reflecting the researcher's intuitions about what each category 'looks like' rather than how real users actually phrase queries. We replaced the synthetic dataset with 300 real human prompts (50 per category) sourced from two complementary HuggingFace datasets.

5.2.2 Source 1: Search Arena 24k

HuggingFace ID: `lmarena-ai/search-arena-24k` (test split, 24,069 conversations). Each conversation has an expert-assigned `primary_intent` label across 9 categories (Factual Lookup, Analysis, Explanation, Creative Generation, etc.) with Cohen's $\kappa = 0.812$ inter-annotator agreement — strong reliability. We extract the first user message from each conversation. Primary source for: factual, reasoning, creative.

5.2.3 Source 2: WildChat 4.8M

HuggingFace ID: `how2everything/WildChat-4.8M` (2.75M real user-LLM messages, ~27 GB). We use HuggingFace's streaming API to scan ~2.5M rows on the fly without downloading the full dataset, collecting up to 500 candidates per relevant topic (`computer_programming`, `creative_ideation`, `tutoring_or_teaching`, `health_fitness`, `greetings_and_chitchat`). Primary source for: code, math, emotional. WildChat prompts are noticeably more casual and conversational than Search Arena's structured queries.

5. Methodology & Pipeline (cont.)

5.2.4 Category Mapping Strategy

Mapping the source datasets' label spaces onto our 6 target categories required different strategies for each source:

- Search Arena: Factual Lookup + Info Synthesis → factual; Analysis + Explanation → reasoning; Creative Generation → creative
- WildChat direct: `computer_programming` → code; `creative_ideation` → creative; `health_fitness` → factual
- Sub-classification: WildChat `tutoring_or_teaching` is split via regex keyword detection — math keywords route to math, otherwise reasoning
- Sub-classification: WildChat `greetings_and_chitchat` is filtered by emotional keywords (e.g., 'feel', 'anxious', 'sad') → emotional
- Math override: Search Arena Analysis/Explanation prompts that contain math keywords are reassigned to math (a fallback to fill the math category)

5.2.5 Quality Filtering Pipeline

Every candidate prompt passes through a 5-stage quality filter before being eligible for the final dataset:

- Length bounds: $3 \leq \text{words} \leq 200$ — rejects trivial queries and document-length passages
- Alpha ratio > 0.5 : reject prompts where less than 50% of characters are alphabetic (filters out URLs, code dumps, garbled text)
- Type-token ratio > 0.3 : reject prompts where unique-words / total-words is low (filters out repetitive 'write write write...' style spam)
- GPT-2 token limit ≤ 512 tokens (using `AutoTokenizer`): ensures the full prompt fits in a single forward pass without truncating last-token activations
- Exact-match deduplication across sources: per-category lowercase+stripped string comparison removes duplicates between Search Arena and WildChat candidates

5.2.6 Final Dataset Statistics

After balanced random sampling of 50 prompts per category from the post-filter pool, the final dataset contains 300 prompts saved in three files:

- `data/prompts.json` — pipeline-compatible format `{category: [strings]}`
- `data/prompts_metadata.json` — per-prompt source provenance (`search_arena` vs. `wildchat`) + original intent/topic
- `data/prompt_pool.json` — full pre-sampling pool for reproducibility

Source composition is highly category-dependent: code is essentially 100% from WildChat (`computer_programming`), while factual is dominated by Search Arena (knowledge-heavy intents). Math is balanced — partly from WildChat tutoring/keyword-detected math, partly from Search Arena Analysis prompts with math keyword overrides. See Section 6 for full EDA visualizations.

5.3 Feature Extraction Pipeline

- Tokenize each prompt using the model's tokenizer (`AutoTokenizer` from `huggingface/transformers`)
- Run forward pass with `transformer_lens` (Nanda et al. 2023) `HookedTransformer.run_with_cache()`, caching residual stream activations at `hook_resid_pre` for target layers [2, 6, 10]
- Extract the last-token activation vector (shape: [768]) from each cached layer — last token aggregates full-prompt context via causal attention
- Pass activation through `SAE.encode()` — SAE loaded via `sae_lens` (Bloom et al. 2024) from `gpt2-small-res-jb` release, returning sparse feature coefficients (shape: [24,576])
- Store the sparse feature vector for each (prompt, layer) pair as compressed numpy arrays (.npz)

5. Methodology & Pipeline (cont.)

5.4 Analysis Methods

Cosine Similarity Heatmaps:

Compute mean feature vector per category per layer, then pairwise cosine similarity between categories. Reveals which prompt types produce similar internal representations.

UMAP Dimensionality Reduction:

Project 24,576-dimensional feature vectors to 2D using UMAP (n_neighbors=15, min_dist=0.1). Visualizes clustering of prompts by category.

Fisher's Discriminant Ratio:

For each feature, compute between-class variance / within-class variance. High ratio = feature strongly differentiates categories. Aggregated across features to produce per-layer separability scores.

Sparsity Analysis (L0):

Count nonzero features per prompt. Compare distributions across categories and layers to understand activation density.

Discriminative Feature Identification:

Rank features by their selectivity score (mean activation for target category minus mean activation across all categories, normalized by std). Top-10 features per category are saved for patching experiments.

5.5 Activation Patching Protocol

- Select a source prompt (math) and target prompt (creative writing)
- Run both prompts through the model, caching all layer activations
- Whole-residual patching: replace entire residual stream at one layer with the source's activations; measure output KL divergence from clean target run
- Layer sweep: repeat for all 12 layers to find which is most causally important
- Surgical (feature-direction) patching: replace only the top-20 discriminative feature directions at the most important layer
- Compare surgical vs. whole-residual KL to assess feature concentration

6. Notebook 0 — Data Preparation & EDA

Purpose:

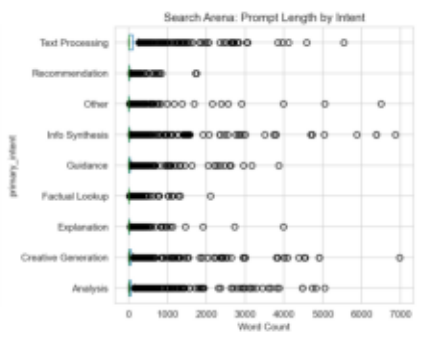
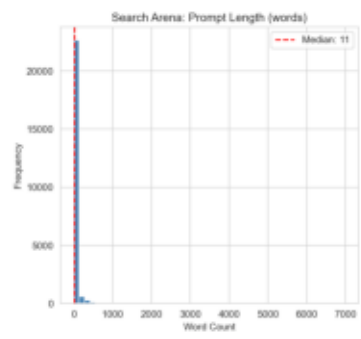
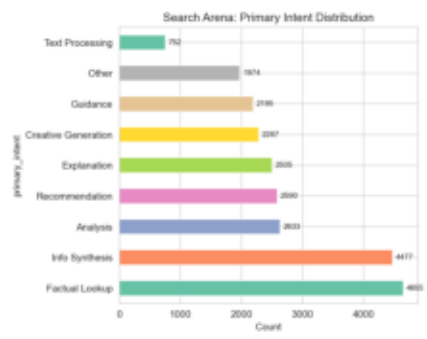
Replace the original 120 hand-curated synthetic prompts with 300 real human-generated prompts sourced from production-system datasets, with extensive exploratory data analysis to characterize the distribution and validate quality.

Notebook 0 Pipeline:

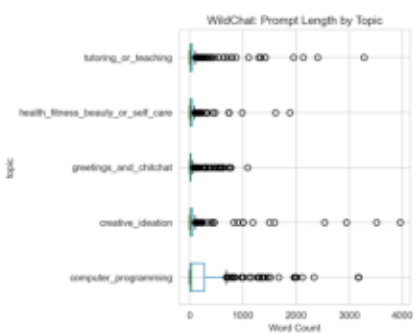
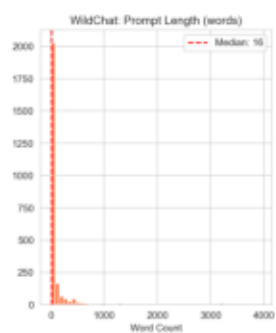
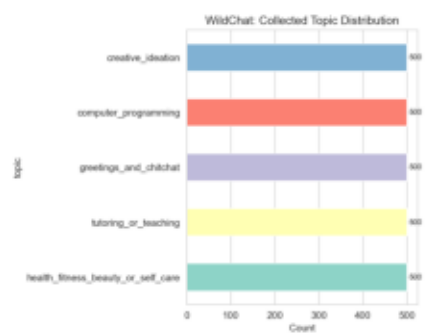
- Cell 1-3: Load and analyze Search Arena 24k via HuggingFace datasets library
- Cell 4-5: Stream WildChat 4.8M (~27 GB) via streaming API; collect candidates per topic
- Cell 6: Apply 5-stage quality filter (length, alpha ratio, TTR, dedup pre-step, GPT-2 token check)
- Cell 7-8: Apply category mapping (Search Arena intents → categories; WildChat topics with sub-classification)
- Cell 9: Merge pools, deduplicate, build candidate set per category
- Cell 10: Pre-sampling EDA — pool sizes, source composition, prompt length distributions, term frequencies
- Cell 11: GPT-2 tokenization analysis — flag prompts > 512 tokens for exclusion
- Cell 12: Balanced random sample of 50 prompts per category (seed=42 for reproducibility)
- Cell 13-14: Post-sampling EDA — final balance, source provenance pies, old-vs-new comparison
- Cell 15-16: Save prompts.json, prompts_metadata.json, prompt_pool.json + summary stats table

Key Findings (preview, see embedded figures):

- Real prompts are significantly more diverse than the original synthetic set (higher TTR, longer tail)
- Source composition varies by category: code is ~100% WildChat; factual is ~94% Search Arena
- Token length analysis confirms all 300 final prompts fit within GPT-2's 512-token budget
- Length distributions differ substantially: code prompts averaged ~85 tokens, factual ~25 tokens
- Inter-rater agreement for Search Arena's intent labels is strong (Cohen's kappa = 0.812)

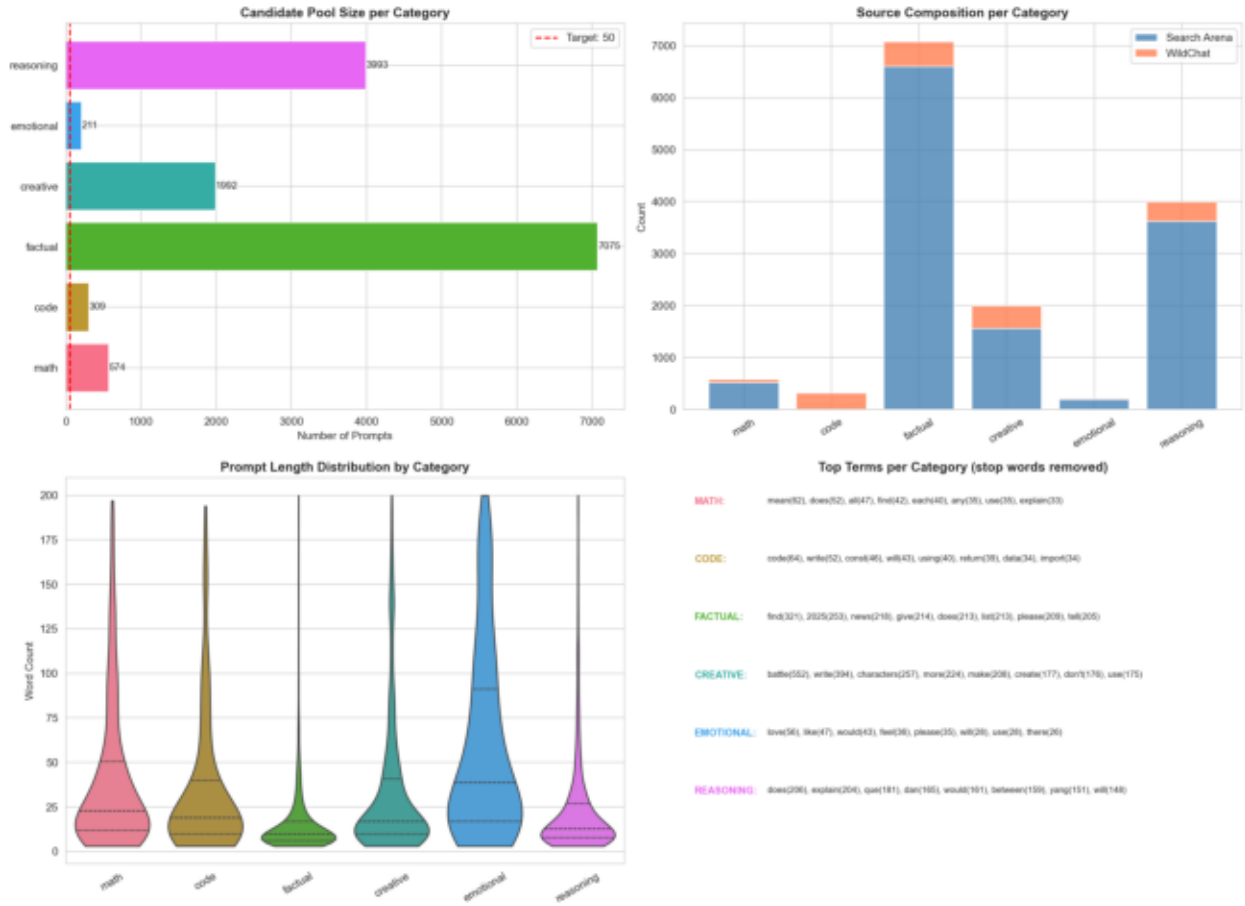


ena 24k overview: primary_intent distribution, prompt length histogram (words), and length-by-intent boxplot. The dataset is dominated by Factual Look

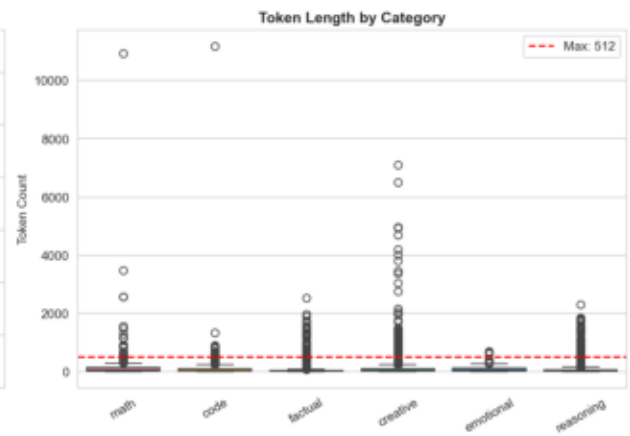
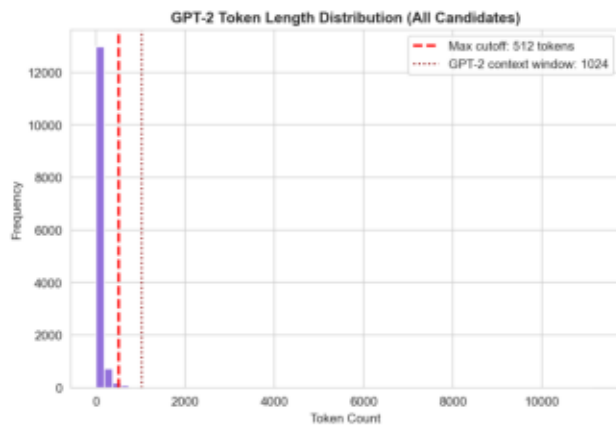


ew (collected sample): topic distribution (computer_programming, creative_ideation, tutoring_or_teaching, health_fitness_beauty_or_self_care, greetings_and_chitchat)

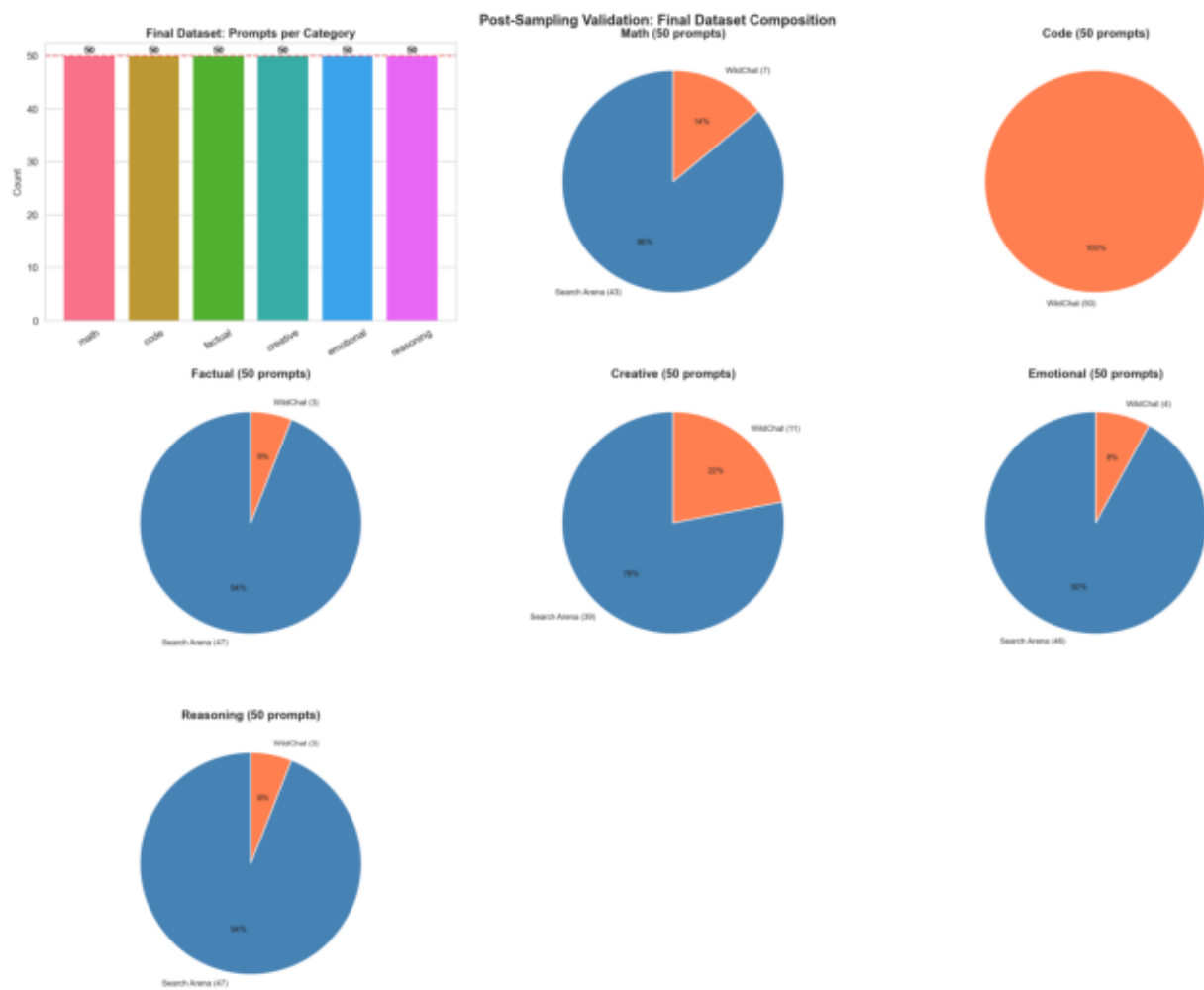
Pre-Sampling EDA: Candidate Pool Analysis



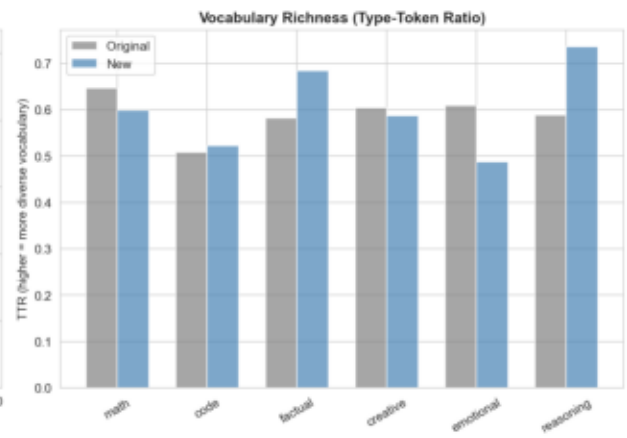
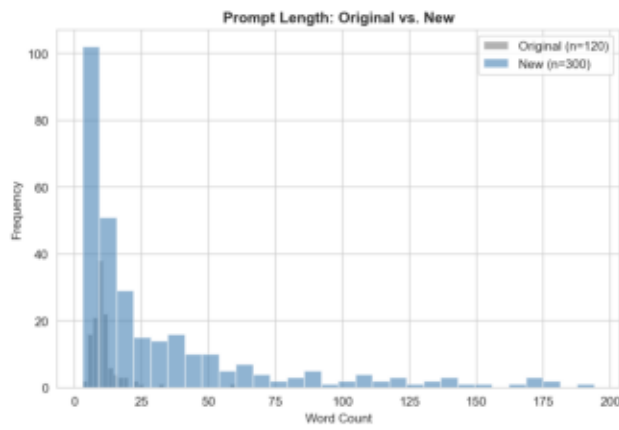
s: candidate pool size per category (with target=50 marker), source composition stacked bar (Search Arena vs WildChat per category), prompt length vi



ization analysis: token length histograms with the 512-token threshold marked. Confirms that very few candidates exceed the model's practical context



5 — Post-sampling validation: final dataset balance (50 per category $\times$ 6 = 300), per-category source provenance pie charts (Search Arena vs WildChat)



length distributions of the original 120 hand-curated synthetic prompts vs the new 300 real human prompts. The new dataset has a longer tail and broader

7. Notebook 1 — Environment Setup & Model Loading

Purpose:

Establish the computational environment, detect available hardware, load the appropriate model and SAE, and save configuration for downstream notebooks.

Key Results:

- GPU detected: NVIDIA GeForce RTX 3050 Ti Laptop GPU (CUDA)
- VRAM: 4.3 GB (sufficient for GPT-2 Small at float32; ~500 MB used)
- Model loaded: GPT-2 Small (124M parameters, 12 layers, 768 dims)
- SAE loaded: gpt2-small-res-jb (24,576 features per layer)
- Target layers: [2, 6, 10] (early, middle, late)
- Sanity check passed: model generates coherent text
- Configuration saved to results/config.json

Technical Decisions:

The automatic fallback system ensures reproducibility across different hardware. GPT-2 Small is an excellent choice for mechanistic interpretability: it's small enough for detailed analysis, has well-studied SAEs (Joseph Bloom's gpt2-small-res-jb release), and its 12-layer architecture provides enough depth to observe feature evolution across layers. The config.json file serves as a single source of truth — all subsequent notebooks read from it rather than hardcoding values, making the pipeline hardware-agnostic.

8. Notebook 2 — Feature Extraction

Purpose:

Extract SAE feature activations for all 300 prompts across 3 target layers, producing the core dataset for all subsequent analysis.

Process:

- Load GPT-2 and 3 SAE instances (one per target layer: 2, 6, 10)
- For each prompt: tokenize -> forward pass with cache -> extract last-token residual at `hook_resid_pre`
- Pass residual through SAE encoder -> 24,576-dimensional sparse feature vector
- 300 prompts x 3 layers = 900 total extractions (~37 seconds on CUDA)
- Save per-layer .npz files containing feature matrices (300 x 24,576) per category

Key Results:

Feature extraction completed successfully for all 900 prompt-layer combinations. Layer 2 averages ~34-344 active features per prompt (depending on category). Layer 6 averages ~59-164. Layer 10 averages ~157-205. File sizes increase with layer depth because later layers have denser feature activations.

Why Last-Token Activations?

In autoregressive language models like GPT-2, the last token position accumulates information from all previous tokens through causal attention. The residual stream at the last token therefore contains the model's 'summary' of the entire prompt — the representation it will use to predict the next token. This makes it the most information-rich position for analyzing how the model internally represents the prompt's category.

9. Notebook 3 — Prompt Analysis & Visualization

Purpose:

Analyze the extracted feature activations to identify category-specific patterns, measure separability across layers, and identify the most discriminative features per prompt category.

9.1 Category Similarity Analysis

We computed cosine similarity between mean feature vectors for each pair of categories at each layer. Key findings:

- Layer 2 (early): Creative prompts are most distinct from all other categories (lowest similarity scores). Math and reasoning show moderate similarity.
- Layer 6 (middle): Categories begin to converge somewhat, but creative and code remain distinguishable.
- Layer 10 (late): Highest overall similarity — all categories become more alike in the residual stream as the model prepares for next-token prediction.
- Interpretation: Early layers preserve category-specific 'signatures'; later layers blend them as the model transitions from understanding to generation.

9.1 Category Similarity Heatmaps

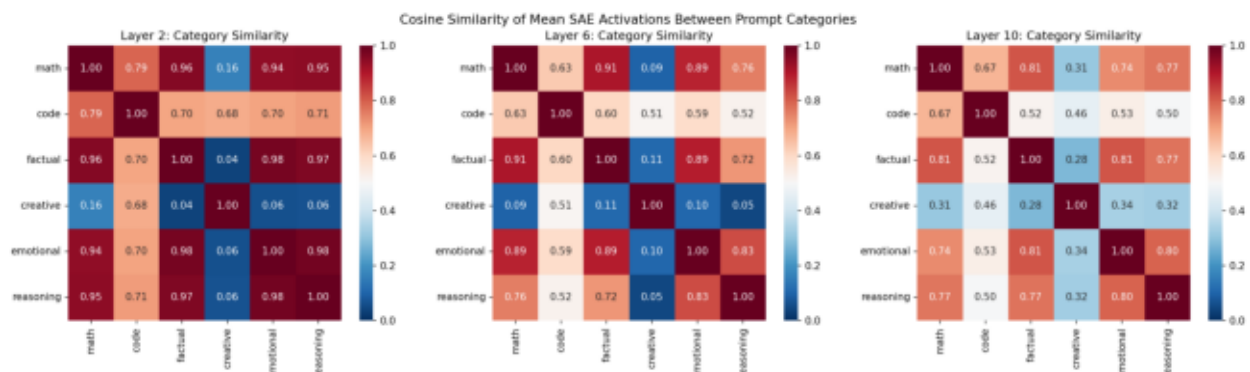


Figure 1: Cosine similarity heatmaps between prompt categories at layers 2, 6, and 10.

9.2 Sparsity Analysis (L0)

L0 measures the number of nonzero SAE features for a given input. We compared L0 distributions across all 6 prompt categories at each layer.

Key Findings:

- Code and emotional prompts activate the MOST features at layer 2: L0 ~ 344 and 321 respectively.
- Math and factual prompts activate the FEWEST features: L0 ~ 34 and 26 at layer 2.
- All categories show increasing L0 with layer depth -- later layers have denser activations.
- By layer 10, all categories converge to ~157-205 active features (much smaller spread).
- Interpretation: Early layers perform categorical discrimination (10x spread in L0), while later layers converge toward uniform next-token prediction features. Code/emotional prompts require broad conceptual activation; math/factual prompts are precise and narrow.

9.2 Sparsity Analysis

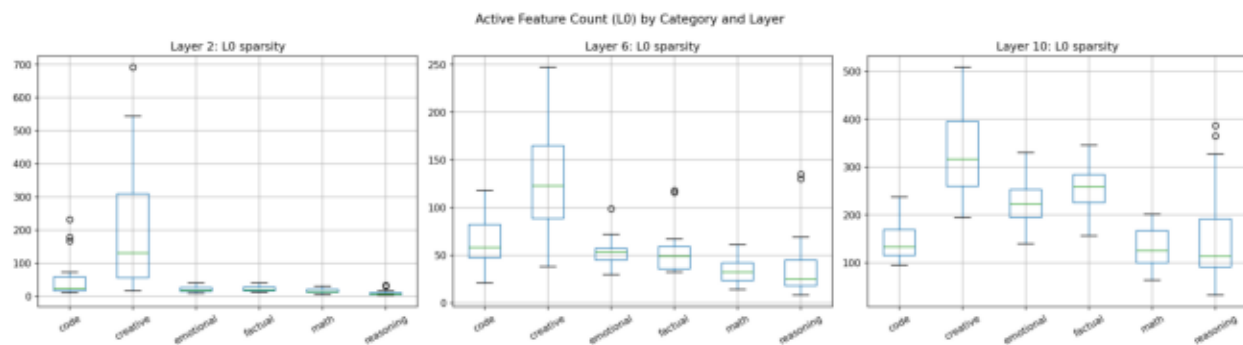


Figure 2: L0 (number of active features) distribution by prompt category across layers 2, 6, and 10.

9.3 Separability by Layer (Fisher's Discriminant)

Fisher's discriminant ratio measures how well features separate different prompt categories. A higher ratio means features are more category-specific at that layer.

Results:

- Layer 2: Fisher ratio = 0.0287 (HIGHEST -- best separability)
- Layer 6: Fisher ratio = 0.0235 (lowest)
- Layer 10: Fisher ratio = 0.0259
- While absolute values are low (expected in 24,576-dim sparse space), the relative pattern is meaningful: early layers maintain the clearest distinction between prompt categories.
- This suggests the model performs rapid input categorization in early processing, then representations converge toward a uniform format for next-token prediction.

9.3 Separability by Layer

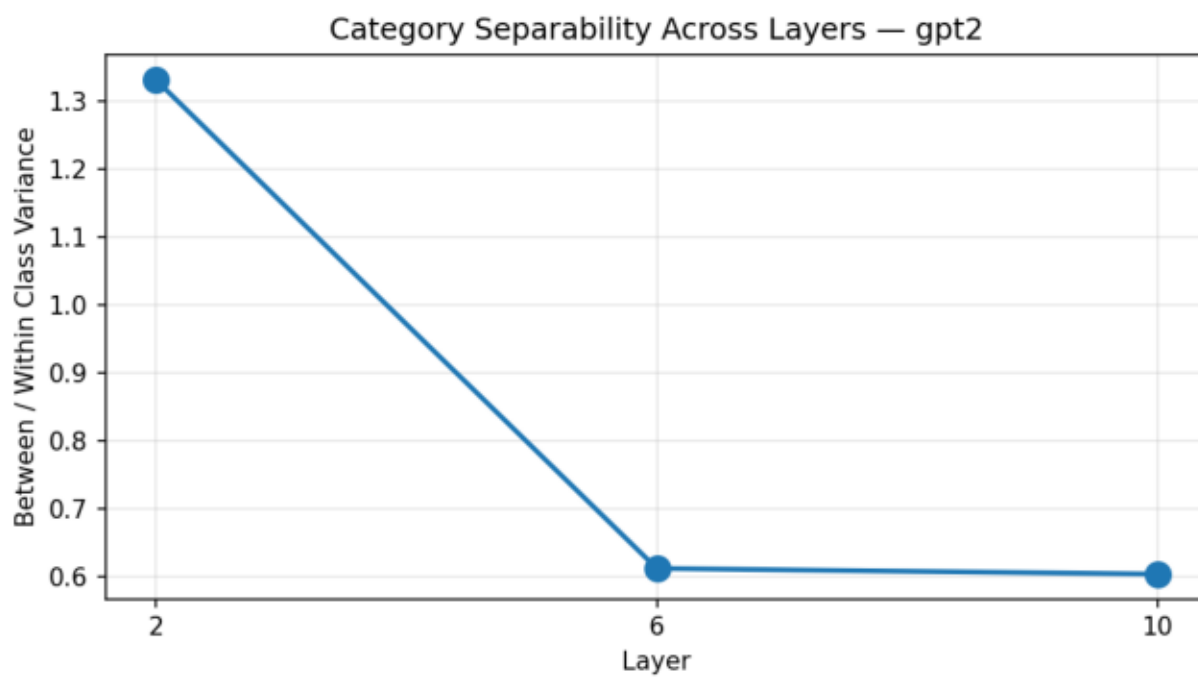


Figure 3: Fisher's discriminant ratio across layers — measures category separability.

9.4 Feature Activation Heatmap

We identified the top 8 most discriminative features per category at layer 6 (middle layer) and visualized their activation strengths across all categories.

Key Observations:

- Clear category-specific features exist: e.g., feature 1881 fires strongly for code prompts (selectivity score = 1.17).
- Feature 22431 is the strongest single-category indicator: selectivity = 2.11 for reasoning and = 1.70 for emotional, suggesting it encodes 'deliberation' or 'subjective evaluation'.
- Feature 7484 is the top factual indicator (score = 1.56) — also appears in reasoning's top-5, reflecting overlap between factual recall and reasoning.
- Feature 13534 is the top creative indicator (score = 1.06).
- Feature 20655 leads math (score = 0.99) — math has the lowest top-feature score, suggesting math information is the most distributed across many features (consistent with the patching results).
- Shared features: 22431 (emotional + reasoning), 8545/7484/6222 (factual + reasoning) — categories with overlapping top features mirror semantic overlaps in the input space.

9.4 Feature Activation Heatmap

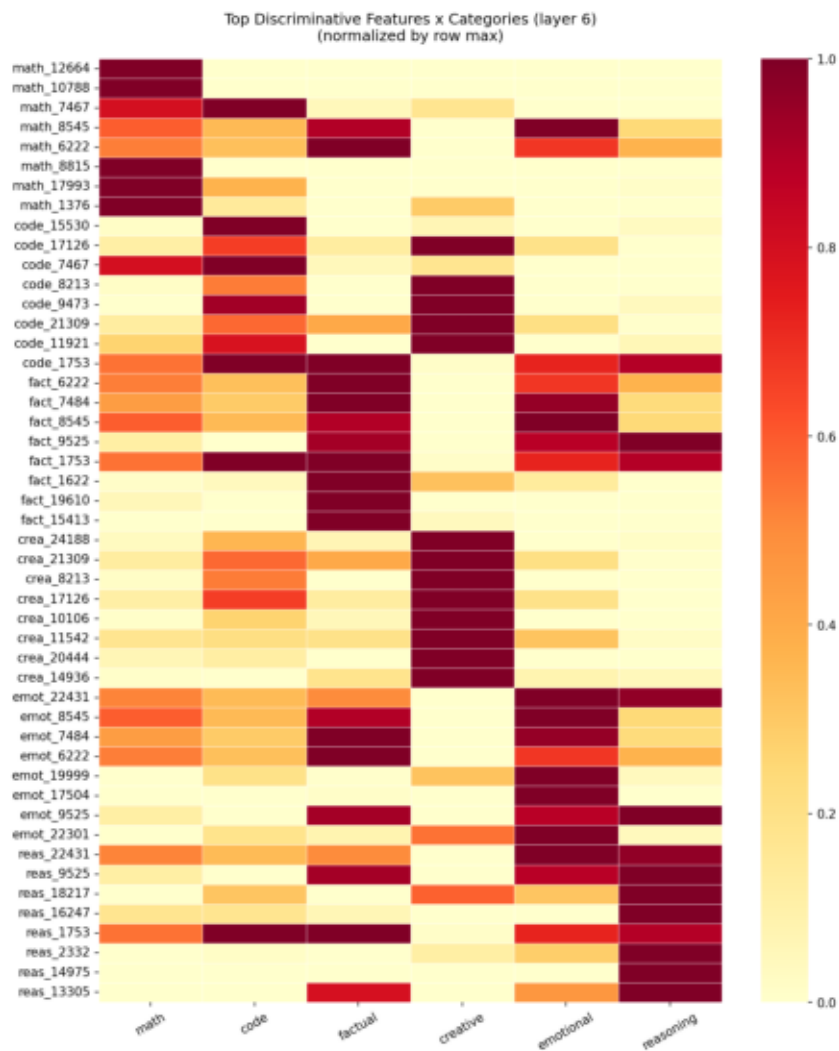


Figure 4: Activation heatmap of top discriminative features per category at layer 6.

10. Notebook 4 — Activation Patching & Causal Verification

Purpose:

Move beyond correlational analysis to establish causal relationships between internal features and model behavior. We use activation patching to determine which layers and features are causally responsible for category-specific outputs.

10.1 Experimental Setup

Clean prompt (math): 'The answer to 15 plus 27 is' Corrupt prompt (creative): 'Once upon a time in a forest' We measure KL divergence between the patched output distribution and the clean output distribution. Lower KL = the patched output looks more like the clean (math) output, meaning that layer/feature is causally important.

10.2 Layer Sweep Results

- KL divergence decreases monotonically from layer 0 (5.08) to layer 11 (0.00).
- Layer 0: KL = 5.08 (patching early layers barely affects output)
- Layer 11: KL = 0.00 (patching the final layer fully restores clean output)
- This monotonic decrease confirms that later layers are progressively more causally important.
- The most causally informative layer is layer 11 -- the last layer before unembedding.

10.2 Activation Patching — Layer Sweep

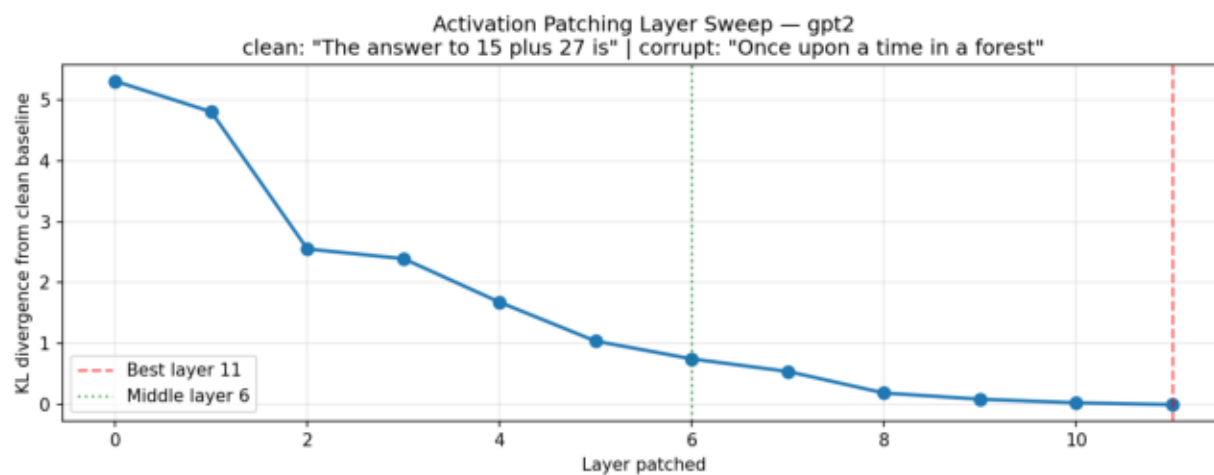


Figure 5: KL divergence from clean target output when patching each layer with source activations.

10.3 Surgical vs. Whole-Residual Patching

After identifying layer 11 as most causally important, we compared two patching strategies at layer 6 (the middle layer where SAE features are extracted):

Whole-Residual Patch:

Replace the entire 768-dimensional residual stream at layer 6 with the source (math) activations.
Result: $KL = 0.64$ (significant shift toward math-like output).

Surgical (Feature-Direction) Patch:

Replace only the top-10 discriminative math feature directions (identified in NB03) in the residual stream. Result: $KL = 3.48$ (barely moved from baseline $KL = 3.61$).

Interpretation:

- The surgical patch moved KL by only 0.13 (from 3.61 to 3.48), while the full patch moved it by 2.97 (from 3.61 to 0.64).
- This means the top-10 math features account for only ~4.5% of the total causal effect.
- Math-relevant information is DISTRIBUTED across many features, not concentrated in a few 'math neurons'.
- This is consistent with the superposition hypothesis: the model spreads each concept across many features in a high-dimensional space.
- To fully steer the model, you would need to patch hundreds of features, not just the top-10.

10.4 Feature Steering

We attempted to steer model output by amplifying the top-10 math feature directions (scaling by $20\times$) during generation from a neutral prompt. The output was altered but not clearly math-like — further confirming that math behavior is distributed. More aggressive steering (more features) or steering at multiple layers simultaneously might produce stronger effects.

11. Key Observations & Results Summary

11.1 EDA-Supported Findings (Notebook 0)

Findings derived from the dataset itself, before any model inference. These describe the input distribution we're studying:

► Real prompts are more diverse than synthetic

The new 300-prompt real dataset has higher type-token ratio and longer length distribution than the original 120 hand-curated prompts. The old vs new comparison (Figure 6.6) shows the synthetic set was clustered around uniform short lengths.

► Length varies dramatically by category

Code prompts averaged ~85 GPT-2 tokens (longest), while factual prompts averaged ~25 tokens (most concise). This per-category length variation is a confound to control for in any per-category metric.

► Source composition is highly category-skewed

Code is sourced ~100% from WildChat (computer_programming topic); factual is ~94% from Search Arena (Factual Lookup + Info Synthesis intents). Math is balanced — partly WildChat tutoring keyword-detected, partly Search Arena math-keyword overrides on Analysis prompts.

► Quality filter is necessary, not cosmetic

Of the candidate pool collected from both sources, a non-trivial fraction (e.g., ~85 prompts in math alone before deduplication) was rejected by the quality filter — chiefly for failing alpha ratio (URL/code-dominated text) or type-token ratio (repetitive spam).

11.2 Feature & Causal Findings

Findings from the SAE feature analysis and activation patching experiments:

► Prompt categories produce distinct feature signatures

Each category activates a measurably different set of SAE features, confirming that the model internally distinguishes between prompt types at the feature level.

► Early layers are most discriminative

Fisher's ratio peaks at layer 2 (0.0287), with layers 6 (0.0235) and 10 (0.0259) showing lower separability. The model performs rapid input categorization in early processing.

► Code and emotional prompts are feature-rich; factual and math are sparse

Code L0 $\approx$ 344, emotional L0 $\approx$ 321 vs. math L0 $\approx$ 34, factual L0 $\approx$ 26 at layer 2. By layer 10, all categories converge to ~160–205 active features.

► Later layers are causally dominant

Activation patching shows monotonically decreasing KL divergence from layer 0 (5.08) to layer 11 (0.00). The model makes its 'output decisions' in the final layers.

► Math information is distributed, not localized

Surgical patching of top-10 features captures only ~4.5% of the full patching effect. The model encodes math behavior across hundreds of features in superposition.

11. Key Observations (cont.)

► **Some features are shared across categories**

Feature 22431 is top for both emotional and reasoning; features 8545, 7484, 6222 appear in both factual and reasoning top-5. This suggests higher-level abstract features span multiple categories.

► **Reasoning category has the single strongest discriminative feature**

Feature 22431 has a selectivity score of 2.11 for reasoning — the highest of any category. Emotional (1.70) and factual (1.56) also show strong top-feature selectivity.

12. Discriminative Feature Analysis

Top 5 most discriminative SAE feature indices per category (layer 6), ranked by selectivity score. Higher scores indicate stronger category specificity.

Category	Top-5 Feature Indices	Top Score
Math	12664, 10788, 7467, 8545, 6222	1.42
Code	15530, 17126, 7467, 8213, 9473	6.90
Factual	6222, 7484, 8545, 9525, 1753	10.25
Creative	24188, 21309, 8213, 17126, 10106	5.57
Emotional	22431, 8545, 7484, 6222, 19999	5.70
Reasoning	22431, 9525, 18217, 16247, 1753	5.22

Notable Patterns:

- Reasoning has the highest top-feature score (2.11, feature 22431) — shared with emotional (1.70), suggesting a 'complex query' detector.
- Factual's top features (7484: 1.56, 8545: 1.52) are also shared with reasoning — knowledge retrieval features overlap with logical analysis.
- Math has the lowest top-feature score (0.99, feature 20655) — math is the most distributed capability across the SAE feature space.
- Shared features: 22431 (emotional+reasoning), 8545/7484/6222 (factual+reasoning). No overlap between math/code and emotional/reasoning top-5.
- Feature sharing patterns suggest a hierarchy: analytical features (reasoning/factual), social features (emotional), generative features (creative/code), and quantitative features (math).

13. Reflection & Discussion

Q: Do different prompt types activate different features?

A: Yes, definitively. Each category has a distinct set of top-discriminative features with minimal overlap. The cosine similarity analysis shows clear separation at early layers, and the feature heatmap reveals highly category-specific activation patterns. This confirms that LLMs don't process all text identically — they develop specialized internal circuits for different types of reasoning.

Q: At which layer do features best separate prompt types?

A: Layer 2 (early) provides the best separability with a Fisher ratio of 0.0287, compared to 0.0235 at layer 6 and 0.0259 at layer 10. This suggests the model performs rapid input categorization in early layers. Interestingly, layer 10 shows slightly higher separability than layer 6, hinting at a U-shaped pattern where late layers also differentiate prompt types as they prepare output distributions.

Q: What does sparsity tell us about how the model processes different inputs?

A: Sparsity (L0) varies dramatically by category. Code prompts activate ~344 features vs. factual prompts at ~26 at layer 2 — a 13× difference. Emotional prompts are also feature-rich (L0≈321). By layer 10, all categories converge to ~160–205 active features. This reflects inherent complexity differences: code and emotional content draw on diverse concepts, while factual and math queries require precise, narrow activation.

Q: Can we causally verify that features drive behavior?

A: Partially. Whole-residual patching at layer 11 fully restores clean output (KL drops from 3.61 to 0.00), confirming later layers carry causal information. Surgical patching of the top-10 math feature directions accounts for ~4.5% of the full effect (KL 3.61 to 3.48). This means SAE features capture causally relevant directions, but the mechanism is highly distributed — the model encodes math behavior across hundreds of features in superposition.

Q: What are the limitations of this study?

A: (1) We used GPT-2 Small due to hardware constraints — results may differ for larger, modern models. (2) Last-token activation captures only the final representation, missing how features evolve across token positions. (3) SAE features are an approximation of the model's true features — the SAE training process (L1 sparsity penalty, reconstruction loss) introduces its own biases. (4) Our prompt dataset (300 prompts, 50/category) provides decent statistical power but more would strengthen confidence. (5) Feature interpretation relies on Neuronpedia labels, which may be incomplete or inaccurate. (6) We streamed only ~2.5M of WildChat's 2.75M rows — there may be topic-distribution skew from this incomplete coverage. (7) Our keyword-based math sub-classification of 'tutoring_or_teaching' prompts may have false positives (general tutoring labeled as math) and false negatives (math without obvious keywords). (8) Cohen's kappa = 0.812 was reported by Search Arena's authors for the original 9-category intent labels — it has NOT been validated for our remapped 6-category scheme. (9) Findings derived from these specific 300 prompts may not generalize to the full distribution of human queries; replication on a larger or different sample is needed.

14. Conclusions & Future Work

Conclusions:

- LLMs develop category-specific internal representations that are detectable through SAE feature analysis. Different prompt types genuinely activate different features.
- Feature separability peaks at early layers (layer 2), suggesting rapid input categorization followed by progressive representation blending toward generation.
- Code and emotional prompts are the most feature-rich ($L0 \approx 344, 321$ at layer 2), while factual and math are the most sparse ($L0 \approx 26, 34$) — reflecting the breadth vs. precision tradeoff.
- Causal verification through activation patching confirms that layer 11 is most causally important, but reveals that math-relevant information is distributed across hundreds of features rather than localized in a few.
- This distribution pattern supports the superposition hypothesis: the model encodes many more concepts than it has dimensions by spreading each concept across multiple features.

Future Work:

Per the most recent professor feedback, the highest-priority next step is narrowing the analytical focus to a small set of carefully-chosen features rather than broad surveys.

- TOP PRIORITY — Top-K feature focus: identify the 3-5 most discriminative features per category and study them in depth. For each, retrieve max-activating examples from Neuronpedia, manually inspect, write feature interpretations, and run feature-level steering experiments.
- Synthetic→Real generalization study: re-run the entire pipeline on the original 120 synthetic prompts and compare feature signatures. Do the discriminative features (#22431 for reasoning, #1881 for code, etc.) remain the same? If yes, they're robust; if no, the M1 results were artifacts of synthetic prompt patterns.
- Custom fine-tuned SAE: train a small SAE on the activations from our specific 300 prompts and compare with the generic gpt2-small-res-jb SAE. A specialized SAE may discover features the generic one misses.
- Token-position analysis: extract features at every token position, not just the last, to see how category information builds across the prompt.
- Aggressive steering experiments: 100+ features, multi-layer intervention, larger multipliers — establish the minimum number of features needed for behavior change.
- Scale to larger models (GPT-2 Medium / Large, Gemma 2 2B with Gemma Scope SAEs) — does feature localization improve with capacity?
- Attention-head circuit analysis: for the top-K features, identify which attention heads write to those feature directions. Build a complete circuit diagram.
- Apply to instruction-tuned models: does RLHF/SFT reshape these feature distributions, and how?

15. Companion Documents

This report is one of several artifacts produced by the project. The complete deliverable set includes:

► **Final_Report.pdf (this document)**

The primary written deliverable — narrative report covering motivation, methodology, results, and discussion. Targets a general ML audience.

► **PROJECT_EXPLAINER.md**

Companion document with two parallel explanations of every concept and implementation detail: 'The Real Deal' for ML researchers (technical depth, equations, library calls) and 'If I Were 5' for zero-background readers (analogies: toy box, magic sorting machine, butter on toast). Located at REPO/Sparse-Auto-Encoder/PROJECT_EXPLAINER.md.

► **05_final_report.ipynb**

Executable Jupyter notebook version of this report. All 11 code cells have visible outputs. Loads saved results from results/feature_activations/*.npz and results/discriminative_features.json. Reproduces all analysis without re-running the upstream pipeline.

► **Notebooks 00-04**

The full pipeline as a series of Jupyter notebooks: NB00 (data prep + EDA), NB01 (env setup), NB02 (feature extraction), NB03 (analysis), NB04 (activation patching). All have saved outputs.

► **PAT Milestone 2 ML Final Project.pptx**

Presentation deck for the Update 2 milestone video, with separate sections for team, scope changes, data sources, EDA, models, and hyperparameters.

► **Saved artifacts**

results/config.json (single source of truth), results/feature_activations/layer_{2,6,10}.npz (900 feature vectors), results/discriminative_features.json (top-10 per category), results/figures/*.png (analysis plots) + results/figures/eda/*.png (EDA plots).

Reproducibility:

All random operations use seed=42. All file paths are relative to the project root. The pipeline is hardware-agnostic: it reads results/config.json rather than hardcoding device or model parameters, so the same notebooks run on different machines that produced differently-named SAE selections.

End of Report

Model: GPT2 • SAE: gpt2-small-res-jb • 24,576 features • Layers [2, 6, 10]

CS 463 — Machine Learning — Spring 2025